

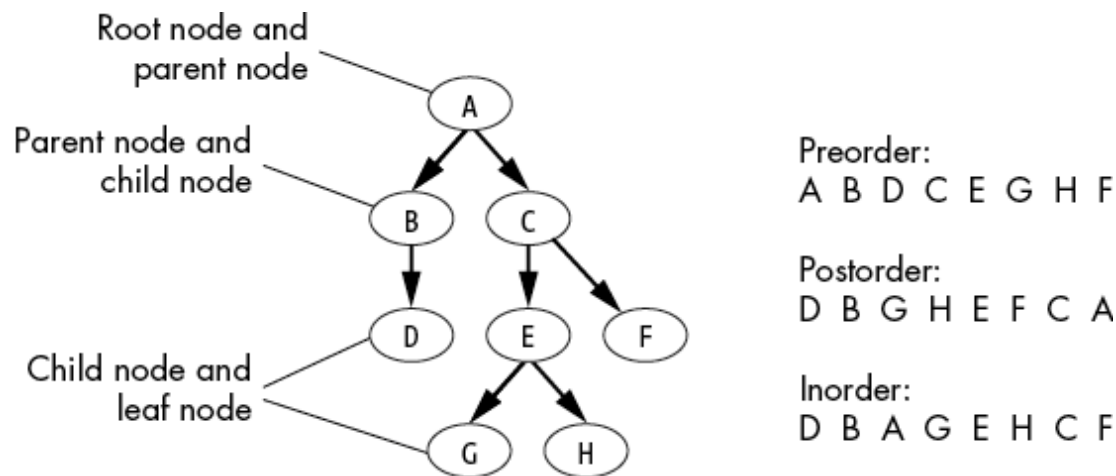


Figure 4-2: A tree with root A and leaves D, G, H, and F, along with its traversal orders

Each node in the tree contains a piece of data (a string of a letter from A to H) and a list of its child nodes. The preorder, postorder, and inorder information in [Figure 4-2](#) is explained in subsequent sections.

In the code for this tree, each node is represented by a Python dictionary (or JavaScript object) with a key `data` that stores the node’s data, and a key `children` that has a list of other nodes. I use the `root` and `node2` to `node8` variables to store each node and make the code more readable, but they aren’t required. The following Python/JavaScript code is equivalent to the previous code listing, though harder for humans to read:

```

root = {'data': 'A', 'children': [{'data': 'B', 'children':
[{'data': 'D', 'children': []}]}, {'data': 'C', 'children':
[{'data': 'E', 'children': [{'data': 'G', 'children': []},
{'data': 'H', 'children': []}]}, {'data': 'F', 'children': []}]}]}
  
```

The tree in [Figure 4-2](#) is a specific kind of data structure called a *directed acyclic graph (DAG)*. In mathematics and computer science, a *graph* is a collection of nodes and edges, and a tree is a kind of graph. The graph is *directed* because its edges have one direction: from parent to child node. Edges in a DAG are not undirected—that is, bidirectional. (Trees in general do not have this restriction and can have edges in both directions, including from a child node back to its parent node.) The graph is *acyclic* because there are no loops, or *cycles*, from child nodes to their own ancestor nodes; the “branches” of the tree must keep growing in the same direction.